DV INTERVIEW · GLANCE SHEET 6

# The WRAP Burst, Step by Step

An INCR burst that lives inside a fixed, aligned window and loops back to the window's start when it reaches the end.

```
Window = (LEN+1) × 2^SIZE
Lower = INT(Start / Window) × Window
Upper = Lower + Window → jump to Lower
```

## 1 Why WRAP exists: critical word first

- A CPU misses in its cache on one word, e.g. `0x14`, and has to fetch the whole cache line `0x00–0x1F`.
- With INCR it would have to start at 0x00 and wait for 0x14 to arrive 6th.
- With WRAP it starts at **0x14** — the word the CPU is stalled on comes back **first** — then the burst wraps round to fill the rest of the line.
- Result: lower miss latency with a single burst, and the line is still fetched completely.

**One-line answer:** WRAP fetches a whole aligned block but in an order that starts at the address you actually need.

## 2 Rules of a legal WRAP

| # | Rule | Reason |
|---|---|---|
| 1 | Length = **2, 4, 8 or 16** beats (AxLEN = 1, 3, 7, 15). | Window must be a power of two to be self-aligned. |
| 2 | Start address **aligned to SIZE** (Start % 2^SIZE = 0). | Every beat must fit exactly in the window. |
| 3 | 2^SIZE ≤ data bus width. | Same as any burst. |
| 4 | AxBURST = `2'b10`. | — |

**Never crosses 4 KB.** Max window = 16 × 128 B = 2 KB, and it's aligned to its own size, so it always sits inside one 4 KB page — no 4 KB check needed.

## 3 The algorithm — 4 steps you can do on a whiteboard

**Step 1 · Window size**
`W = (LEN+1) × 2^SIZE`
e.g. 8 beats × 4 B = 32 B

**Step 2 · Lower boundary**
`L = INT(Start / W) × W`
(= Start with low bits cleared)

**Step 3 · Upper boundary**
`U = L + W`
first address *outside* the window

**Step 4 · Walk**
add 2^SIZE each beat; when address = U, set it to L. Stop after LEN+1 beats.

REFERENCE MODEL (SYSTEMVERILOG)

```
function automatic void wrap_addrs(
    input  bit [31:0] start, input bit [7:0] len,
    input  bit [2:0]  size,  ref bit [31:0] q[$]);
  int unsigned nb    = 1 << size;
  int unsigned win   = nb * (len + 1);
  bit [31:0]   lower = (start / win) * win;
  bit [31:0]   upper = lower + win;
  bit [31:0]   a     = start;
  repeat (len + 1) begin
    q.push_back(a);
    a += nb;
    if (a == upper) a = lower;
  end
endfunction
```

BIT-TRICK VERSION (WINDOW IS A POWER OF 2)

```
mask  = W - 1;              // e.g. 32 B → 0x1F
lower = start & ~mask;
next  = lower | ((addr + nb) & mask);
```

Only the low log2(W) bits move; upper bits stay fixed. That's exactly how RTL address generators implement WRAP — no comparator needed.

**Shortcut:** the number of beats before the wrap = (U − Start) / 2^SIZE. Beats after the wrap = the rest.

**Special case:** if Start = L, the burst never wraps and is identical to INCR.

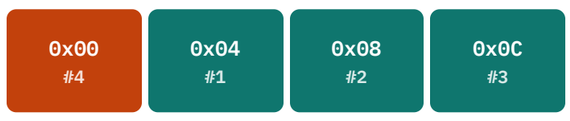
## 4 One window, every starting point — 4 beats × 4 B (window 16 B, 0x00–0x0F)

START 0X00 (= INCR)




No wrap.

START 0X04




04 → 08 → 0C → **00**

START 0X08




08 → 0C → **00** → **04**

START 0X0C




0C → **00** → **04** → **08**

Teal = before the wrap, orange = after. The **same 16 bytes** are always covered — only the order changes, rotating to start at the requested word.

## 5 More worked examples

| Case | LEN / beats | SIZE | Start | Window | L → U | Address sequence |
|---|---|---|---|---|---|---|
| **A** · 2-beat | 1 / 2 | 8 B | `0x38` | 16 B | 0x30 → 0x40 | `0x38, 0x30` |
| **B** · cache line | 7 / 8 | 4 B | `0x14` | 32 B | 0x00 → 0x20 | `0x14, 0x18, 0x1C,` **wrap** `0x00, 0x04, 0x08, 0x0C, 0x10` |
| **C** · late start | 7 / 8 | 8 B | `0x1F8` | 64 B | 0x1C0 → 0x200 | `0x1F8,` **wrap** `0x1C0, 0x1C8, 0x1D0, 0x1D8, 0x1E0, 0x1E8, 0x1F0` |
| **D** · 16-beat | 15 / 16 | 4 B | `0x2C` | 64 B | 0x00 → 0x40 | `0x2C, 0x30, 0x34, 0x38, 0x3C,` **wrap** `0x00, 0x04 … 0x24, 0x28` (11 more) |
| **E** · high address | 3 / 4 | 16 B | `0x8020` | 64 B | 0x8000 → 0x8040 | `0x8020, 0x8030,` **wrap** `0x8000, 0x8010` |

NARROW WRAP · 64-BIT BUS · SIZE 4 B · 4 BEATS · START 0X108

Window 16 B → L = 0x100, U = 0x110 → `0x108, 0x10C, 0x100, 0x104`




WRAP VS INCR, SAME REQUEST

| Start 0x14 · 8 × 4 B | Sequence | Bytes covered |
|---|---|---|
| **INCR** | `0x14 … 0x30` | 0x14–0x33 (spills into next line) |
| **WRAP** | `0x14 … 0x1C, 0x00 … 0x10` | 0x00–0x1F (exactly one line) |

Same LEN/SIZE, same first beat — but WRAP stays inside the aligned 32 B line while INCR runs past it.

## ✗ Illegal WRAPs

| Request | Why illegal |
|---|---|
| LEN = 5 (6 beats) | 6 is not 2/4/8/16 |
| LEN = 0 (1 beat) | Minimum WRAP length is 2 |
| Start 0x06, SIZE 4 B | Start not aligned to 4 |
| LEN = 31 in AXI4 | WRAP max is 16 (only INCR grew to 256) |
| SIZE 8 B on a 32-bit bus | Beat wider than the bus |

## ✓ Verifying WRAP

```
constraint c_wrap {
  burst == WRAP -> len inside {1, 3, 7, 15};
  burst == WRAP -> addr % (1 << size) == 0;
}
```

- Scoreboard uses `wrap_addrs()` to know which byte each beat hits.
- Cover: **every start offset in the window** (start = L, mid, last slot) × len {2,4,8,16} × size.
- Cover narrow WRAP and WRAP at max size.
- Negative tests (if DUT must flag them): unaligned start, bad LEN → expect SLVERR or assertion.
- Assertion: every WRAP beat address stays in [L, U).

## ? WRAP interview Q&A

| Question | Crisp answer |
|---|---|
| What is a WRAP burst? | INCR that loops back to the start of an aligned window of size (LEN+1)×2^SIZE. |
| Main use? | Cache line fill — critical word first. |
| Allowed lengths? | 2, 4, 8, 16 beats. |
| Why must the length be a power of 2? | So the window is self-aligned and address bits wrap naturally. |
| Can WRAP cross 4 KB? | No — window ≤ 2 KB and aligned to its size. |
| Can WRAP start unaligned? | No — must be aligned to 2^SIZE. |
| WRAP 4 × 4 B from 0x2C? | Window 0x20–0x2F → 0x2C, 0x20, 0x24, 0x28. |
| WRAP 8 × 2 B from 0x0A? | Window 0x00–0x0F → 0x0A, 0x0C, 0x0E, 0x00, 0x02, 0x04, 0x06, 0x08. |
| When is WRAP identical to INCR? | When the start address is the lower boundary. |
| How does RTL generate the next address? | Increment only the low log2(window) bits; upper bits fixed. |
| Does WRAP change WSTRB rules? | No — lanes follow each beat's address, as for INCR. |